

add the option `-E`. Let's run this on our test file:

```
[manish@clone ~]$ grep -E '[a-z]{11}' testfile
to carry out few regular expressions
```

Here, 'expressions' is the only all-lowercase 11-character string in the file; so this is the only line printed as the output.

There are quite a few character classes that are very commonly used in regular expressions, and these are provided as named classes. For example, the '[a-z]' class of lower-case alphabets that we used above, has the named class '[:lower:]'. Naturally, '[:upper:]' is upper-case letters A to Z, and '[:alpha:]' is all alphabetic characters, equivalent to '[:lower:]' plus '[:upper:]'. '[:digit:]' is the digits 0 to 9, and '[:alnum:]' is alphanumeric characters—a combination of '[:alpha:]' and '[:digit:]'. The Grep manual, linked above, lists out more of these named classes.

When a caret (^) is used as the first character in a character class, it is a negation of the class, effectively meaning, 'none of these characters'.

Line and word anchors

The ^ anchor specifies that the pattern following it should be at the start of the line:

```
[manish@clone ~]$ grep '^th' testfile
this
```

The \$ anchor specifies that the pattern before it should be at the end of the line.

```
[manish@clone ~]$ grep 'is$' testfile

Hi
```

The operator < anchors the pattern to the start of a word.

```
[manish@clone ~]$ grep '\<efe' testfile

to carry out few regular expressions
```

Similarly, > anchors the pattern to the end of a word.

```
[manish@clone ~]$ grep 'le>' testfile
is test file
```

The \b (word boundary) anchor can be used in place of < and > to signify the beginning or end of a word:

```
[manish@clone ~]$ grep -e '\breg' testfile
to carry out few regular expressions
```

Finally, we look at the | (alternation) operator, which is part of the extended regex features. A pattern containing this operator separately matches the parts on either side of it; if either one is found, the line containing it is a match. The parts can themselves be complex regular expressions, so this means you can check

each line in a file for multiple search patterns in one pass.

```
[manish@clone ~]$ grep -E 'hi|bc' testfile

this

Abcd
```

That was pretty simple; so let's try a more complicated one. Can you reason out why the output lines for this regex are as shown below?

```
[manish@clone ~]$ grep -E '^([t-z])+([a-z]+)' testfile

this

to carry out few regular expressions

123 456

ABCD
```

Using shell expansions in the pattern input to Grep

As mentioned earlier, if you don't single-quote the pattern passed to Grep, the shell could perform shell expansion on the pattern and actually feed a changed pattern to Grep. This can also be done intentionally, when you need it—let's look at a few examples.

```
[manish@clone ~]$ grep "$HOME" /etc/passwd
root:x:0:0:root:/root:/bin/bash
operator:x:11:0:operator:/root:/sbin/nologin
```

Here, we intentionally use double quotes to make the Bash shell replace the environment variable \$HOME with the actual value of the variable (in this case, '/root'). Thus, Grep searches the passwd file for the text '/root', yielding the two lines that match.

```
[manish@clone ~]$ grep 'whoami' /etc/passwd
root:x:0:0:root:/root:/bin/bash
operator:x:11:0:operator:/root:/sbin/nologin
```

Here, back-tick expansion is done by the shell, replacing 'whoami' with the user name ('root') that is returned by the whoami command.

Well, we hope this has set you on your way to using this very efficient tool.  **END**

By: Manish Rane

The author is an IT consultant, trainer and hacktivist on FOSS technologies, and currently works as a network security lead at an MNC. With almost a decade's administrative and network security experience, he firmly believes Linux to be the future. His interests include cyber forensics and investigations, and he also works as a forensic consultant on open source technologies. In his free time he loves to hack around Linux.